

Web Application Development

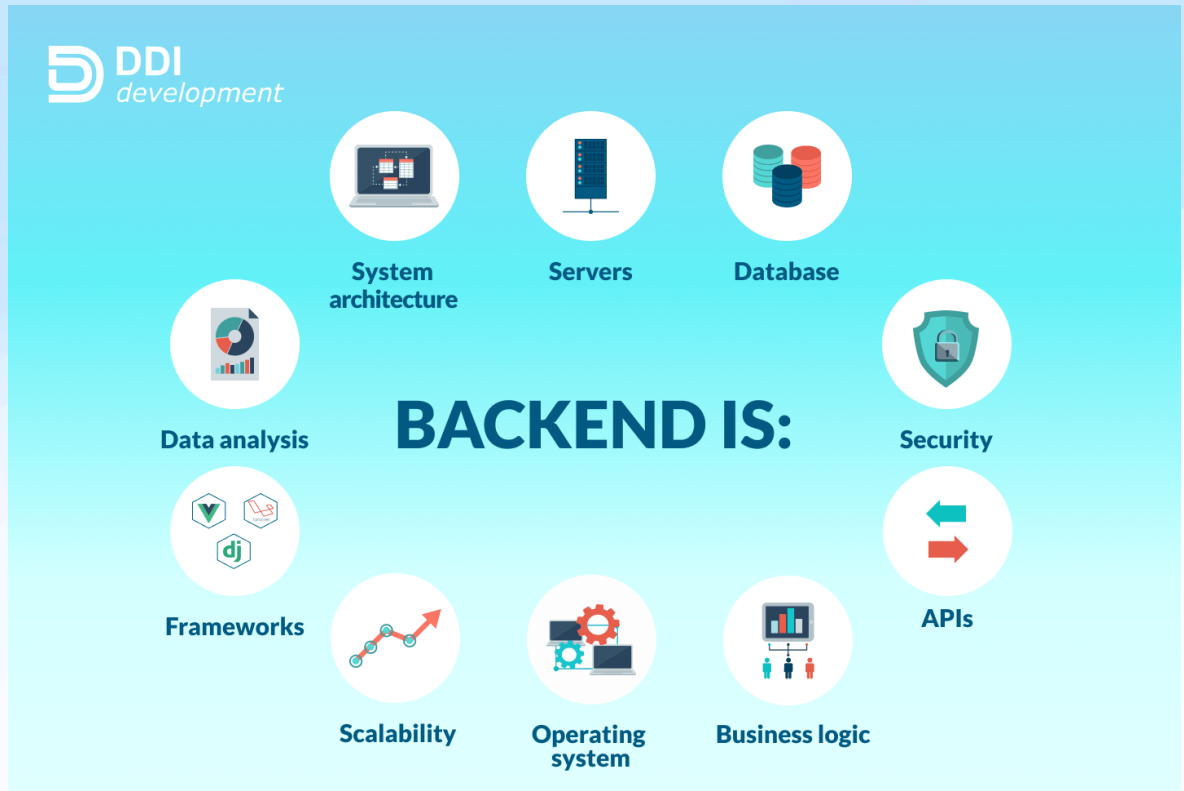
❖ Node.js and Express.js

Ammar Ahmad Khan

Backend Development

Backend development refers to **server-side development**, where the focus is on how the website or web application works. Unlike the frontend, which **users interact** with directly, the backend involves:

- Business logic
- Database interactions
- Authentication
- Server configuration
- API design



1. Business Logic

- **What it is:** The rules and logic that define how your application behaves.
- **Example:** If a user books a seat on a bus, business logic checks if the seat is available and calculates the fare.
- **In Code:** It's implemented through functions, conditions, and algorithms in your backend.

2. Database Interactions

- **What it is:** Communicating with a database to **store, retrieve, update, or delete** data.
- **Example:** When a user logs in, the backend checks the username and password from the database.
- **Tools Used:** SQL (MySQL, PostgreSQL), NoSQL (MongoDB), ORMs (Mongoose, Sequelize)

3. Authentication

- **What it is:** The process of verifying **who** a user is.
- **Example:** Login system using username/password, OAuth (Google Login), or JWT tokens.
- **Goal:** Ensure only authorized users access protected parts of the application.

4. Server Configuration

- **What it is:** Setting up the `server` to run your backend properly.
- **Example Tasks:**
 - Setting up ports (e.g., `app.listen(3000)`)
 - Handling environment variables
 - Enabling CORS
 - Setting timeout, compression, or HTTPS
- **Tools:** `dotenv`, `cors`, `helmet`, and server settings

5. API Design

- **What it is:** Defining how clients (frontend, mobile apps) communicate with your backend.
- **APIs (Application Programming Interfaces)** expose routes (URLs) that handle data.
- **Example:**
 - GET /users – Fetch all users
 - POST /login – Submit login form
 - PUT /profile/123 – Update user profile
- **Standards Used:** RESTful APIs, GraphQL

Introduction to Server-Side Development

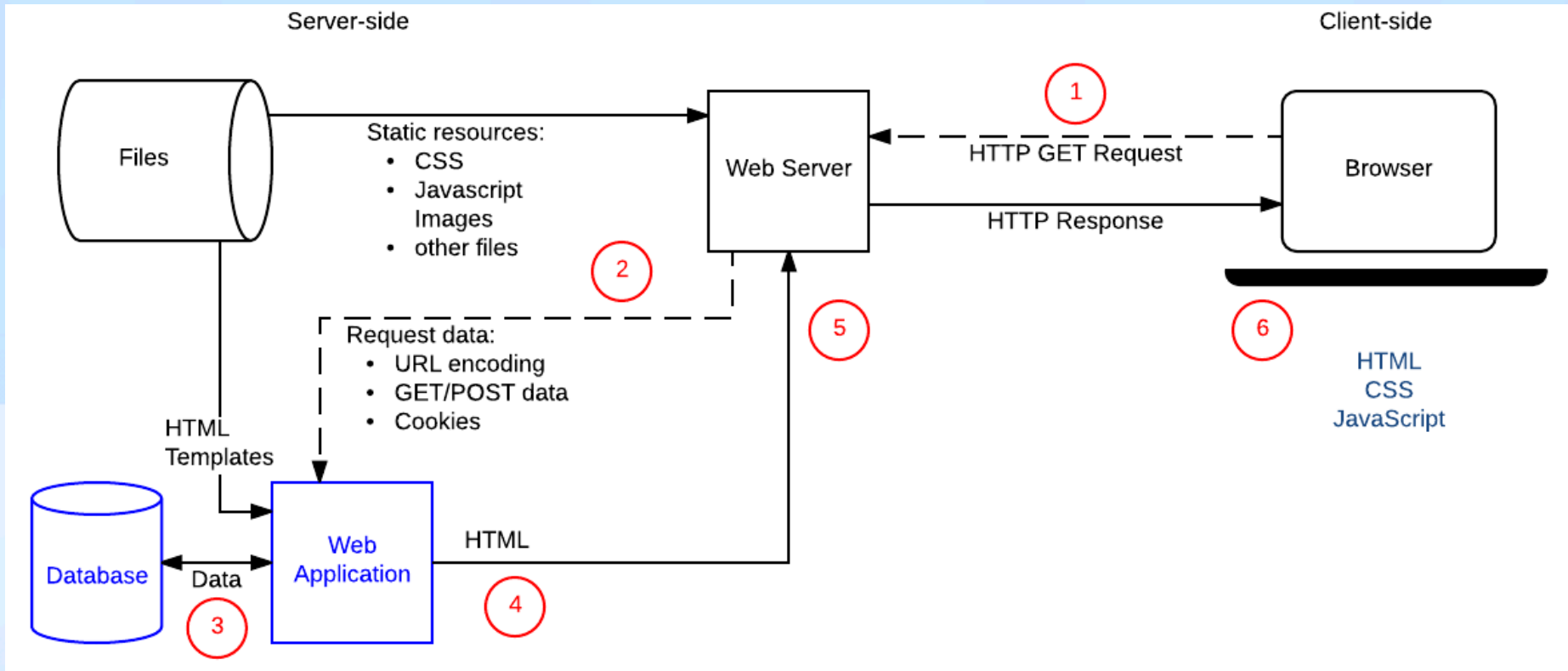
What is Server-Side Development?

Server-side development means building the **part** of a web application that runs on the **server**, not the **client** (browser). It processes client requests, performs operations like **reading/writing** data to a database, and sends back the appropriate response.

Typical Flow:

1. Client (browser) sends a request (e.g., form submission).
2. Server receives and processes the request.
3. If needed, it fetches data from a database.
4. Server sends back the response (e.g., a web page or JSON data).

Server-side Diagram



Node.js – JavaScript on the Server

Node.js is an open-source, cross-platform **runtime** environment that allows you to run **JavaScript on the server**.

Key Features:

- Built on Chrome's **V8 JavaScript engine**
- **Non-blocking I/O** and **event-driven** architecture for high performance
- Ideal for building **real-time applications** (e.g., chat apps, REST APIs)
- Uses **npm (Node Package Manager)** to manage packages/libraries

1. Built on Chrome's V8 JavaScript Engine

- **What it means:** Node.js uses the **same engine** that Google Chrome uses to run JavaScript: it's called **V8**.
- **Why it's good:** V8 is fast and optimized for performance, so your backend **JavaScript** code runs very efficiently.
- **Example:** Server-side code executes as quickly as frontend code in Chrome.

2. Non-blocking I/O and Event-Driven Architecture

- **What it means:** Node.js doesn't wait (or block) while performing tasks like [reading files or querying a database](#). Instead, it uses **callbacks or events** to continue working on other tasks.
- **Why it's useful:** It allows Node.js to handle **many requests at once**, making it **scalable**.
- **Example:** While one user's file is being uploaded, the server can respond to another user's request without delay.

3. Ideal for Real-Time Applications

- **Why:** Its **event-driven** nature and support for **WebSockets** make Node.js perfect for apps where users need instant updates.
- **Examples:**
 - Chat applications
 - Online multiplayer games
 - Real-time dashboards
 - Collaborative tools like Google Docs

4. Uses npm (Node Package Manager)

- **What it is:** npm is a tool that comes with Node.js. It lets you **install and manage packages (libraries)**.
- **Why it matters:** It has over **1 million packages**, so you rarely need to write common features from scratch.
- **Example Packages:**
 - express – framework for building web apps
 - mongoose – MongoDB object modeling
 - bcrypt – for password hashing
 - socket.io – for real-time communication

Express.js – Fast and Minimal Web Framework for Node.js

Express.js is a lightweight and flexible **web application framework** for Node.js. It simplifies routing, middleware integration, and request handling.

Why use Express?

- Minimal setup to get a server up and running
- Easier routing and middleware system
- Large ecosystem and community support

1. Minimal Setup to Get a Server Up and Running

- **What it means:** With just a few lines of code, you can create a fully functioning web server using Express.
- **Why it's useful:** You can start developing fast: no need to write complex boilerplate code like you would with plain Node.js.

```
const express = require('express');
```

```
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send('Hello, Express!');  
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```


2. Easier Routing and Middleware System

- **What is Routing?** Defining URL paths and what to do when users visit them (e.g., /login, /products).
- **What is Middleware?** Functions that run **before** your route logic: e.g., to check if a user is logged in, parse JSON, or handle errors.

3. Large Ecosystem and Community Support

What it means: Express has been around for a long time and is widely used. You'll find:

- Thousands of **middleware packages** (for auth, logging, CORS, etc.)
- Huge community help (Stack Overflow, GitHub)
- Rich learning resources (YouTube, documentation, tutorials)

Example Tools:

- body-parser – parse request bodies (now part of Express)
- morgan – logging HTTP requests
- cors – handle cross-origin requests

Node

Previously: Javascript code is only run and used in browser. Like in **console window** of a browser or a Javascript file separately.

So, it cannot be run **outside** the browser.

In **browser** we actually have an **interpreter** which runs the javascript code. So, browser is an **environment** for JS code to run.

Node(History)

Python and other languages are used to run or **make** web servers.

In **2009**, Node.js is **released**-> a popular technology that is used to **build backend** or **webservices**.

Node.js is **purely** based on **javascript**->famously used for server side **programming**.

Node.js is not a language, library or framework.

Node Installation

Go to **terminal** and **search** if already installed or **not**:

Search “**node**”

Or to only check version “**node -v**”

If we get a prompt “**Command not found**” then we need to install it.

Go to nodejs.org and install it.

Interesting Fact: VScode is also made using nodejs

Node REPL

REPL(Read, Evaluate, Print, and Loop)

It is same as we have a **console window** in **browser**, we have a **REPL** window in **system**.

To open the window just write “**node**” on terminal.

Now terminal command will **not** be **entertained** so **ls** and **pwd** will not work here because we are now in a **node world**.

Node Command:

```
console.log(“hello world”)
```

```
.help
```

```
.exit
```

Creating File in Node

Step 1. Open Visual Studio Code

Open your project folder or create a new one.

Step 2. Create a New File

Command: `touch script.js`

Step3. Write any code inside that file

Code:

```
let n = 5;  
  
for (let i = 0; i < n; i++) {  
  console.log("hello", i);  
}
```

Step 4. Now run the code not in browser but in terminal: `node script.js`

Architecture of Node.js

Node.js is based on the "**Single-threaded Event Loop**" architecture, which is designed for **asynchronous and non-blocking I/O operations**.

1. **Single-threaded:**

- Unlike traditional multi-threaded servers (like Apache), Node.js uses a single thread to handle multiple clients.

2. **Event-driven:**

- Actions are triggered via events and listeners (e.g., when a request comes in, a specific callback handles it).

3. **Non-blocking I/O:**

- It performs I/O operations like reading from disk or database asynchronously, allowing other operations to continue while waiting for results.

4. **Libuv:**

- A C library that provides the event loop and handles asynchronous I/O.

Callbacks

A callback function is a **function** that is passed as an **argument** to another function and is executed **after** the **completion** of some operations.

This mechanism allows JavaScript to perform tasks like **reading files**, **making HTTP requests**, or **waiting** for user input **without** blocking the execution of the program.

Example Comparison: Single Threaded

Multi-threaded server (e.g., Apache):

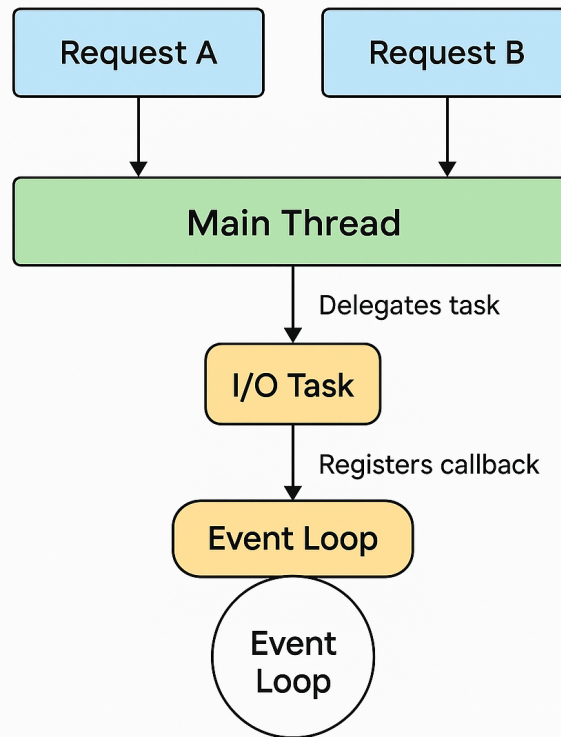
1. Request A comes in → Thread 1 handles it.
2. Request B comes in → Thread 2 handles it.
3. Each thread may block while waiting (e.g., for DB).

Node.js (Single-threaded):

1. Request A comes in → Handled on the **main thread**.
2. If it needs to wait (e.g., DB), it **delegates** the task to the system and continues.
3. Request B comes in → Handled on the **same thread**.
4. When DB response is ready, **callback** is triggered via the **event loop**.

Diagram: Single Threaded

Node.js: Single-threaded



What Does "Event-driven" Mean in Node.js?

Event-driven programming means that **code execution is determined by events**: such as user actions, messages, or I/O results: rather than a fixed sequence of instructions.

In **Node.js**, this is a core concept. It uses an **event-driven architecture** where:

- The system emits an **event**
- A **listener (callback function)** responds when that event occurs

Example:

Imagine you're waiting for a food delivery:

- Instead of sitting at the door (blocking), you continue watching TV (doing other tasks).
- When the doorbell rings (an event), you respond (listener/callback).

Core Features of Node.js

- **Asynchronous and Event-Driven:** All APIs are non-blocking.
- **Fast Execution:** Powered by Google's V8 JavaScript engine.
- **Single Programming Language:** Uses JavaScript on both server and client side.
- **Scalability:** Suitable for micro-services and scalable network applications.
- **Cross-platform:** Can be used on Windows, Linux, and macOS.

Modules in Node.js

Node.js follows a **modular architecture**, meaning functionality is separated into reusable pieces (modules).

Types of Modules:

- 1. Built-in Modules:** Provided by Node.js (e.g., fs, http, path, events)
- 2. User-defined Modules:** Your own JS files exported using `module.exports`
- 3. Third-party Modules:** Installed via NPM (e.g., express, mongoose)

What is `module.exports` in Node.js?

In Node.js, each file is treated as a **module**. To share code (functions, variables, objects) between files, we use:

`module.exports`

- This defines **what gets exported from a module (file)**.
- You can **import** it in another file using **`require()`**

Ways to Use `module.exports`

- There are **3 common ways**

1. Export a single function or value

// math.js

```
const add = (a, b) => a + b;  
module.exports = add;
```

script.js

```
const add = require('./math');  
console.log("Addition:", add(5, 3));
```


2. Export an object with multiple properties

// maths.js

```
const add = (a, b) => a + b;  
const mul = (a, b) => a * b;  
const pi = 3.1416;
```

```
module.exports = {  
  add,  
  mul,  
  pi  
};
```

// script.js

```
const maths = require('./maths');
```

```
console.log("Addition:", maths.add(5  
, 3));
```

```
console.log("Multiplication:",  
maths.mul(4, 6));
```

```
console.log("Value of PI:",  
maths.pi);
```

3. Assign directly to export (shorthand)

// maths.js

```
exports.add = (a, b) => a + b;
```

```
exports.mul = (a, b) => a * b;
```

```
exports.pi = 3.1416;
```

Script.js: Same as previous

Real Working Example

File Structure:

project/

├── script.js ✓ Main file

└── maths.js ✓ Module with exports

math.js

// **maths.js**

const add = (a, b) => a + b;

const sub = (a, b) => a - b;

const mul = (a, b) => a * b;

const pi = 3.1416;

module.exports = {

add,

sub,

mul,

pi

};

script.js

```
// script.js
```

```
const maths = require('./maths');
```

```
console.log("Addition: ", maths.add(5, 3));
```

```
console.log("Subtraction: ", maths.sub(10, 4));
```

```
console.log("Multiplication: ", maths.mul(6, 7));
```

```
console.log("Value of PI: ", maths.pi);
```

Export in Directories

Goal

Suppose you have a structure like this:

```
project/
├── script.js
├── fruits/
│   ├── banana.js
│   ├── apple.js
│   ├── orange.js
│   └── index.js ← (this will collect and export everything)
```

Step 1: fruits/banana.js

// banana.js

```
const bananaInfo = () => {  
  return "Bananas are yellow and full of potassium."  
};
```

```
module.exports = bananaInfo;
```

Step 2: fruits/apple.js

// apple.js

```
const appleInfo = () => {  
  return "Apples are red, green, or yellow and great for health.";  
};
```

```
module.exports = appleInfo;
```


Step 3: fruits/orange.js

```
// orange.js
const orangeInfo = () => {
  return "Oranges are citrus fruits and rich in vitamin C.";
};

module.exports = orangeInfo;
```

Step 4: fruits/index.js (Central export)

This file **gathers all fruits and exports them** as a single module.

```
// index.js
```

```
const banana = require('./banana');
```

```
const apple = require('./apple');
```

```
const orange = require('./orange');
```

```
module.exports = {
```

```
  banana,
```

```
  apple,
```

```
  orange
```

```
};
```

Step 5: script.js (Importing from fruits directory)

```
// script.js
```

```
const fruits = require('./fruits'); // It automatically picks index.js
```

```
console.log(fruits.banana()); // Bananas are yellow and full of  
potassium.
```

```
console.log(fruits.apple()); // Apples are red, green, or yellow and great  
for health.
```

```
console.log(fruits.orange()); // Oranges are citrus fruits and rich in  
vitamin C.
```

Practice Tasks

Open file on QOBE named as “Lecture 17-18 Tasks”